

SOSS

Version 1.0

Installation and User Guide

22 September 2026

Contents

1	Introduction	3
2	SOSS Structure	5
2.1	INSTALL	5
2.2	SRC	5
2.3	WORK	6
2.3.1	INPUTS	6
2.3.2	OUTPUTS	7
2.3.3	RUN	7
3	SOSS Computational Modules	8
3.1	ImportSettings	8
3.1.1	ImportSettings Functions	11
3.2	ImportData	12
3.2.1	ImportData Functions	12
3.3	ExportData	12
3.3.1	ExportData Functions	13
3.4	Energy	13
3.4.1	Energy Functions	14
3.5	SelectSolver	14
3.5.1	SelectSolver Functions	15
3.6	SANSS	15
3.6.1	SANSS Functions	23
3.6.2	SANSS Output Files	25
4	Using SOSS	26
4.1	System Requirements	26
4.2	Unpacking the Software	26
4.3	Installation	27
4.4	Validation	27
4.5	Running SOSS	33
4.5.1	Preparing Input Files	33
4.5.2	Executing Scripts	33
4.5.3	Examining Output Files	34

1 Introduction

SOSS (Search for Optimally Stable Structures) is an in-house optimization package developed at BCAM (Basque Center for Applied Mathematics) and written in Julia. It is designed to search for ground-state configurations of solid structures using heuristic optimization techniques. The core algorithm exploits the characteristic features of solid-state ionic structures and processes to reduce a high-dimensional optimization problem into a discrete model with precomputed components of the objective function, enabling efficient evaluations.

The SOSS algorithm proceeds as follows:

1. Reads an input atomic structure file containing N_S lattice sites: N_{Anch} sites occupied by anchored ions (species whose positions remain fixed during optimization), and $N_{\text{Sopt}} = N_S - N_{\text{Anch}}$ sites initially occupied by host ions (species whose positions may change during optimization). The latter are referred to as optimization sites.
2. Identifies the subset of N_{Sopt} optimization sites.
3. Defines the interchangeable ion types¹ (species whose positions may change during optimization) to be distributed among the optimization sites. The interchangeable ions are divided into the aforementioned host ions (species already present in the initial structure) and newly added ions/dopants (species introduced during the optimization process)². The final structure contains N_{Ihost} host ions and N_{Iadd} added ions³, giving a total of $N_{\text{Inter}} = N_{\text{Ihost}} + N_{\text{Iadd}}$ interchangeable ions.
4. Computes the (precomputed) components of the objective function (interaction energy), which essentially consist of arrays storing the interaction energies between the different ion types present in the system while accounting for the possible ion-site combinations. To optimize memory storage and computational efficiency, the interactions are divided into three components: anchored-anchored (a constant scalar value), interchangeable-anchored (a 2D array storing the interaction energies between interchangeable ions and their anchored neighbors), and interchangeable-interchangeable (a 4D array storing the interaction energies between pairs of interchangeable ions). These precomputed components are used during optimization to efficiently evaluate energy differences between configurations, avoiding costly on-the-fly calculations.
5. Initializes a random distribution of N_{Inter} interchangeable ions and $N_V = N_{\text{Sopt}} - N_{\text{Inter}}$ vacancies (empty optimization sites) over the N_{Sopt} optimization sites.

¹Each ion type corresponds to a chemical species; e.g., if a structure contains Li and Ga ions, the Li ions constitute one ion type and the Ga ions constitute another. Each ion type possesses its own physical properties (e.g., Li ions have a charge of +1, whereas Ga ions have a charge of +3), which motivates this classification.

²Being "host" or "added" is a property associated with each interchangeable ion type, which we refer to as its status.

³A fraction of the host ions contained in the input structure is removed ($N_{\text{Sopt}} - N_{\text{Ihost}}$), thereby creating free sites where dopants can be introduced. The condition $N_{\text{Iadd}} \leq N_{\text{Sopt}} - N_{\text{Ihost}}$ must always be satisfied. Attempting to add more dopants than allowed produces an error in the algorithm, since no additional sites are available to accommodate them.

6. Searches for the optimal (lowest-energy) distribution of ions and vacancies over the optimization sites using a heuristic optimization solver.
7. Reconstructs the final optimal structure by combining the N_{Inter} interchangeable ions of the optimized subset with the N_{Anch} anchored ions of the unmodified subset and exports the results into an atomic structure file.

This approach makes **SOSS** particularly well suited for solid-state ionic applications—such as solid oxide fuel cells, solid-state batteries, supercapacitors, various energy storage devices, and sensors—where the lattice sites remain fixed (or nearly fixed) and the primary degrees of freedom arise from the redistribution of specific chemical species (those we have referred to as interchangeable ions).

Key features of the current version of **SOSS** include:

- Input/Output structure format: VASP format (POSCAR).
- Optimization solvers: Simulated Annealing [1] (trajectory-based), tuned to address **SOSS**-specific problems, and enriched with performance enhancing features.
- Objective function: Coulomb interaction energy, computed through (1) a truncated Coulomb potential or (2) an Ewald summation.
- Overlap function (optional): Additional repulsion term in the objective function to avoid undesired ion overlaps.
- Ion-Site constraint (optional): Classification of lattice sites based on their initial chemical element, enabling selective occupation rules for each ion type during optimization.

Although the current implementation of **SOSS** uses Coulomb interaction energy as the objective function and Simulated Annealing as the optimization solver, the **SOSS** algorithm is not restricted to these choices. The **SOSS** package is designed with flexibility in mind and can accommodate any objective function and any metaheuristic optimizer. Its modular architecture facilitates the integration of a broader range of models and solvers in future versions. Moreover, incorporating additional constraints to leverage a priori knowledge about the system under study is also fully supported, further enhancing the applicability of the package to diverse materials problems.

The package has been successfully tested on LLZO ($\text{Li}_7\text{La}_3\text{Zr}_2\text{O}_{12}$) structures with up to 5184 atoms ($3\times 3\times 3$ supercell), both in their pristine composition and in doped variants (e.g., Ga- and Ta-substituted).

For further details on the theoretical background and practical applications of the code, refer to the manuscript “Screening Energetically Stable Structures in Solid-State Ionics Applications”, Alex Teruel et al. [2], included in the **SOSS** package.

2 SOSS Structure

The **SOSS** package is organized into three main directories: **INSTALL**, **SRC**, and **WORK**, as shown in Figure 1. The content of each directory is described below.

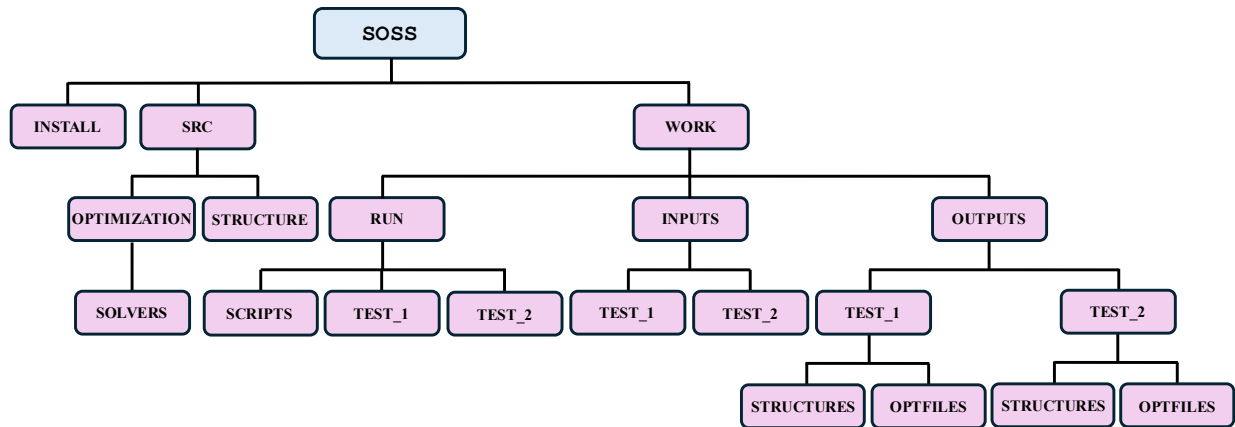


Figure 1: Structure of the **SOSS** package.

2.1 INSTALL

The **INSTALL** directory accommodates all files related to the installation process:

- *info.txt*: Contains the installation-related information for the user.
- *Project.toml*: Provides a list of the required Julia packages.
- *CondaPkg.toml*: Provides a list of the required Python packages.
- *install_soss.jl*: Creates a Julia project environment in the installation directory and imports the Julia and Python packages needed to use **SOSS**.
- *test_installation.jl*: Tests that the Julia and Python packages are properly installed.

2.2 SRC

The **SRC** directory includes the computational modules of the package. It is divided into two subdirectories:

- **STRUCTURE**: Contains files related to import/export data and optimization problem definition:
 - *ImportSettings.jl*: Defines the **ImportSettings** module.
 - *ImportData.jl*: Defines the **ImportData** module.
 - *ExportData.jl*: Defines the **ExportData** module.

- *Energy.jl*: Defines the **Energy** module.
- **OPTIMIZATION**: Contains files related to the optimization solvers:
 - *SelectSolver.jl*: Defines the **SelectSolver** module.
 - **SOLVERS**: Stores files that define each of the optimization solvers.
 - * *SANSS.jl*: Defines the **SANSS** optimization solver module.

Details about each module, their functions, and parameters can be found in section 3.

2.3 WORK

The **WORK** directory is used to prepare and execute jobs, and to store the results. In the distributed version of the **SOSS** package, it is also intended for running validation tests supplied in the **SOSS** package. It provides files needed for preparation and execution of validation tests and organizes the storage of test results. The **WORK** directory includes the **INPUTS**, **OUTPUTS**, and **RUN** subdirectories.

2.3.1 INPUTS

The **INPUTS** directory contains the input files used for the validation tests described in section 4.4, located in `./WORK/INPUTS/TEST_1` and `./WORK/INPUTS/TEST_2`. These files are:

- *instruct.vasp* (required): Input atomic structure file. A file containing the initial structure in VASP format (POSCAR) [3].
- *settings.jl* (required): Settings file. A file containing the optimization settings tags described in section 3.1 (summarized in Table 1).
- *params.jl* (required): Parameters file. A file containing the parameters of the selected optimization solver. Note that the content of this file depends on the selected optimization solver. For **SANSS** optimization solver, this file must contain the parameters described in Table 3.
- *seeds.dat* (optional): Seeds file. A two-column (space-separated) data file specifying the seeds used to initialize the random number generators (RNGs) for each independent optimization run. The first column defines the seed for the RNG responsible for generating the initial random configuration, while the second column defines the seed for the RNG used during the stochastic search. The file must contain at least as many rows as the total number of planned optimization runs. This file is optional; if omitted, the RNGs are initialized with arbitrary seeds.

A new directory in **INPUTS** with the same required files must be created to perform a new optimization (e.g., `./WORK/INPUTS/MY_JOB`). The entire process of preparing a new input file directory, running the job, and examining the results is explained in section 4.5.

2.3.2 OUTPUTS

The **OUTPUTS** directory is initially empty and is populated with results after each optimization. For each completed optimization, a directory with the same name as the corresponding input directory is created automatically. Figure 1 shows the content of the **OUTPUTS** directory after running the provided tests.

Each output directory includes two subdirectories:

- **OPTFILES**: Stores files generated during the optimization process. The specific content depends on the solver used, but these files usually monitor the evolution of the optimization (e.g., energy vs. optimization step). See section 3.6.2 for more information on **SANSS** optimization solver output files.
- **STRUCTURES**: Stores the atomic structure files of the final structures. If multiple optimization cycles/runs are performed, each resulting structure is saved as a separate file named *outstruct.x.vasp*, where *x* is the run number.

2.3.3 RUN

The **RUN** directory contains a **SCRIPTS** subdirectory, where the scripts to run **SOSS** are provided. These scripts are:

- *run_soss.jl*: Coordinates the optimization process. It performs the following steps:
 1. Loads the modules from the **SRC** directory.
 2. Includes the settings file *settings.jl* to the **ImportSettings** module.
 3. Includes the parameters file *params.jl* to the **SelectSolver** module.
 4. Calls the `get_settings()` function from the **ImportSettings** module to generate the settings tuple **Settings** (which contains the optimization settings defined in *settings.jl*).
 5. Calls the `get_data()` function from the **ImportData** module to read the input atomic structure file *instruct.vasp* and generate the data tuple **Data** (which contains the problem instance⁴).
 6. Calls the `solve()` function from the **SelectSolver** module to generate the parameters tuple **Params** (which contains the optimization solver parameters defined in *params.jl*), perform the optimization using the solver specified in *settings.jl*, and return the solution tuple **Solution** (which contains the solution of the optimization problem⁵).

⁴A problem instance is the information that, for a given **Settings** and initial structure, defines the optimization problem. It essentially contains information about the lattice sites, the ions, and the precomputed components of the interaction energy.

⁵A solution of the optimization problem consists of one or more optimized configurations (depending on the number of runs) together with their associated energies.

7. Calls the `save_structure()` function from the `ExportData` module to reconstruct the final structure and write the output atomic structure file(s) *outstruct.x.vasp* ($x = 1, \dots$, number of runs).
- *run_job.sh*: Runs a specific job by taking the name of the inputs directory and the number of CPUs to use as arguments, and calling the script *run_soss.jl*. The results are stored in the corresponding outputs directory. See more details about this script in section 4.5.2.

The **RUN** directory also contains the **TEST_1** and **TEST_2** empty subdirectories, which are intended for running the provided test cases, as explained in section 4.4.

3 S0SS Computational Modules

3.1 ImportSettings

The `ImportSettings` module reads the tags defined in the settings file *settings.jl* (explained below and summarized in Table 1) for a job and generates the corresponding settings named tuple⁶ `Settings`.

These tags characterize different aspects of the optimization problem. First, the `solver`, `IN_format`, and `OUT_format` tags define the optimization solver to be used and the formats of the input and output atomic structure files. (*instruct.vasp*, containing the lattice sites and initial ions, and *outstruct.vasp*, containing the optimized structure). At present, only the **SANSS** solver (described in section 3.6) and the VASP format [3] are available.

The next group of tags defines the material system to be optimized. The `FU_symbols`, `FU_numbers`, `FU_charges`, and `FU_protons` tags specify the chemical species present in the initial structure, the number of ions of each species per formula unit, their charges, and their atomic numbers (number of protons), respectively. The `FU_copies` tag determines how many formula units are contained in the provided input structure (*instruct.vasp*). An important detail is that the input structure may contain "virtual" ions whose only purpose is to define lattice sites for the optimization. An example is provided in the tests described in section 4.4, where the input structure corresponds to LLZO containing 15 Li ions per formula unit, although only 7 are actually occupied in the real material. In such cases, the `FU_numbers` tag must contain the number of physically occupied ions (7 in this example).

After defining the initial structure, the settings related to the ions involved in the optimization are specified. The `Ihost_symbols` tag defines the chemical species acting as host ions, i.e., the species already present in the input structure whose lattice sites will be used during optimization. In practice, these are the species on which substitutions, dopants, and/or vacancies will be introduced. This tag therefore determines the optimization sites of the system. The `Ihost_numbers` tag specifies, for each host ion species, the number of ions per formula unit that the final structure must contain. Then, the tags associated with

⁶According to Julia documentation, named tuples are instances of the `NamedTuple` type, which has two parameters: a tuple of symbols giving the field names, and a tuple type giving the field types. For simplicity, throughout this document, we will refer to named tuples simply as tuples.

added ions (dopants) are defined. The `Iadd_symbols`, `Iadd_numbers`, `Iadd_charges`, and `Iadd_protons` tags specify the chemical species to be added to the material, the number of ions of each species per formula unit, their charges, and their atomic numbers, respectively. In summary, the `Ihost_symbols` and `Ihost_numbers` tags define the optimization sites (corresponding to the positions occupied by the host ions in the input structure) together with the number of host ions in the final structure, whereas the `Iadd_symbols` and `Iadd_numbers` tags define the dopants to be introduced and their quantities. Together, these tags determine both the optimization sites and the chemical species distributed among them.

The remaining system-related tags are `Iadd_sites` and `IonSiteConstraint`, which allow prior knowledge about the material to be incorporated into the optimization process by constraining which host ion sites may be occupied by each added ion species. This mechanism operates as follows. When `IonSiteConstraint = true`, the optimization sites (i.e., the sites occupied by host ions in the initial structure) are divided into subsets, each subset corresponding to one of the host ion species specified in `Ihost_symbols`. Each host ion is naturally assigned to its own subset. The `Iadd_sites` tag then specifies, for each added ion species, which subset of host ion sites it is allowed to occupy, following the same order used in `Ihost_symbols`. In this way, the optimization sites are partitioned into subsets and ions are prevented from occupying sites that are known *a priori* to be physically unfavorable. A practical example of this functionality is provided in **Test 2** of section 4.4. If this feature is not required, it can simply be disabled by setting `IonSiteConstraint = false`; in this case, `Iadd_sites` may contain arbitrary values (e.g., zeros).

When no dopants are introduced, the vectors associated with the tags `Iadd_symbols`, `Iadd_numbers`, `Iadd_charges`, `Iadd_protons`, and `Iadd_sites` should simply be left empty, and `IonSiteConstraint` can be set to false.

Finally, the `LongRange` and `ShortRange` tags define the energy model used during optimization. The former determines how long-range electrostatic interactions are calculated, whereas the latter controls short-range interactions, specifically the overlap term that imposes a minimum allowed distance between ions. These two tags are discussed in more detail in section 3.4, where the implemented energy models are described.

Table 1: SOSS settings tags defined in the settings file *settings.jl*.

Name and Data Type of a tag	Description	Values
<code>solver</code> <code>::String</code>	Optimization solver	“SANSS” ⁷
<code>IN_format</code> <code>::String</code>	Format of the input atomic structure file	“VASP”

Continued on next page

⁷In the current version of SOSS, SANSS is the only implemented optimization solver. The user is responsible for implementing new solvers.

Name and Data Type of a tag	Description	Values
OUT_format ::String	Format of the output atomic structure file	“VASP”
FU_symbols ::Vector{String}	Names of anchored and host ions in initial structure	User’s choice
FU_numbers ::Vector{Float64}	Number of anchored and host ions per formula unit in the initial structure	User’s choice
FU_charges ::Vector{Float64}	Charges of anchored and host ions in the initial structure	User’s choice
FU_protons ::Vector{Int64}	Atomic numbers of anchored and host ions in the initial structure	User’s choice
FU_copies ::Int64	Number of formula units contained in the supercell	User’s choice
Ihost_symbols ::Vector{String}	Names of host ions	User’s choice
Ihost_numbers ::Vector{Float64}	Number of host ions per formula unit in the final structure	User’s choice
Iadd_symbols ::Vector{String}	Names of added ions (dopants)	• Undefined, if no impurities • User’s choice, otherwise
Iadd_numbers ::Vector{Float64}	Number of added ions per formula unit in the final structure	• Undefined, if no impurities • User’s choice, otherwise
Iadd_charges ::Vector{Float64}	Charges of added ions	• Undefined, if no impurities • User’s choice, otherwise
Iadd_protons ::Vector{Int64}	Atomic numbers of added ions	• Undefined, if no impurities • User’s choice, otherwise
Iadd_sites ::Vector{Int64}	Subset of host ion sites (following the order defined in Ihost_symbols) available for added ions	• Vector of 0, if no constraints • User’s choice, otherwise
IonSiteConstraint ::Bool	Boolean variable to control the ion-site constraint functionality	• false, if no constraints • true, otherwise

Continued on next page

Name and Data Type of a tag	Description	Values
LongRange ::Vector{Any}	Long-range interaction energy model vector [LR_model, LR_params], where LR_model is the selected model (Int64) and LR_params are the parameters for that model (Vector{Float64}) Available models: 1. Truncated Coulomb: Electrostatic energy is computed using the Coulomb potential truncated at the specified cutoff distance r_cutoff (in Å) 2. Ewald summation: Electrostatic energy is computed using the Ewald summation method, with all parameters determined automatically	• Truncated Coulomb: LR_model = 1 LR_params = [r_cutoff] Recommended values r_cutoff = 7.0 Å • Ewald summation: LR_model = 2 LR_params = [] (empty)
ShortRange ::Vector{Any}	Short-range interaction energy model vector [SR_model, SR_params], where LR_model is the selected model (Int64) and LR_params are the parameters for that model (Vector{Float64}) Available models: 1. Overlap: A strong positive repulsive energy is applied to interchangeable ions within a distance r_overlap (in Å) of another ion to exclude those configurations	• No interaction: SR_model = 0 SR_params = [] (empty) • Overlap repulsion: SR_model = 1 SR_params = [r_overlap]

3.1.1 ImportSettings Functions

- `get_EnergyModelTags()`: Reads the LongRange and ShortRange tags and reformats them to be added to Settings. It is called by `get_settings_with_dopant()` and `get_settings_without_dopant()`.
- `get_settings_with_dopant()`: Reads *settings.jl* and creates Settings when dopants are added.
- `get_settings_without_dopant()`: Reads *settings.jl* and creates Settings when no dopants are added.

- `get_settings()`: Picks and calls either `get_settings_without_dopant()` or `get_settings_with_dopant()`, and returns **Settings**.

3.2 ImportData

The **ImportData** module reads the settings tuple **Settings**, the input atomic structure file in VASP format [3] for a job, *instruct.vasp*, and generates the data tuple **Data** required for optimization. This module uses the Energy module (section 3.4) to generate the precomputed components of the interaction energy, which are stored in **Data**.

3.2.1 ImportData Functions

- `read_structure()`: Reads the initial structure contained in *instruct.vasp*.
- `get_species()`: Gets the chemical elements that appear in the initial structure (host and anchored ion types).
- `get_protons()`: Gets the atomic number of each host ion type.
- `get_charges()`: Gets charges of anchored and host ions.
- `check_QConservation()`: Verifies charge conservation.
- `get_OptSites()`: Identifies N_{Sopt} optimization sites. If ion-site constraint is defined in the settings file *settings.jl*, the optimization sites are classified into subsets (as explained in section 3.1).
- `get_IonTypes()`: Supplies information about interchangeable ions belonging to each ion type, such as their total number, chemical species, atomic number, charge, host or added status, corresponding optimization sites subset (optional). When ion-site constraint is used, each host ion type is assigned to the subset defined by itself (by definition), and each added ion type is assigned to the subset established in the **Iadd_sites** tag in *settings.jl* (see Table 1).
- `get_data()`: Reads **Settings** and, using the functions above, generates and returns **Data**, which contains information about the atomic structure, optimization sites, ion types, and precomputed energy components required to optimize the system and export the results.

3.3 ExportData

The **ExportData** module generates the output atomic structure file(s) for a job in VASP format, *outstruct.x.vasp* ($x = 1, \dots$, number of runs of the job). The input of this module includes the settings tuple **Settings** (see section 3.1), the data tuple **Data** (see section 3.2), and the solution tuple **Solution** (see section 3.5).

3.3.1 ExportData Functions

- `get_structure()`: Combines the final (optimal) distribution of interchangeable ions among the optimization sites with the anchored ions, which remain in their initial positions in the unmodified subset of sites, to obtain the final (optimal) atomic structure.
- `write_poscar()`: Writes atomic structure files in VASP format.
- `save_structure()`: Reads **Settings**, **Data**, and **Solution**, gets the final structure, and generates the output structures *outstruct.x.vasp*.

3.4 Energy

The **Energy** module generates the precomputed components of the interaction energy (in eV) based on the energy model specified in *settings.jl* via the **LongRange** and **ShortRange** tags (see Table 1).

The total interaction energy is calculated as a sum of a long-range potential (**LongRange**) and an optional short-range potential (**ShortRange**) energies. In the current implementation, there are two long-range models and one short-range model.

Long-range interaction:

- Truncated Coulomb: Electrostatic energy is computed using a Coulomb potential truncated at a specified cutoff radius, `r_cutoff` (in Å). Exact electrostatic interactions are calculated for distances up to `r_cutoff`, beyond which interactions are neglected. This model is generally preferred for non-periodic systems. To use this model, set **LongRange** = [1, `r_cutoff`] in the settings file.
- Ewald Summation: Electrostatic energy is computed using the Ewald summation method [4], with all parameters determined automatically, using the `EwaldSummation` class within the `pymatgen.core.ewald` module of the Python Materials Genomics (`pymatgen`) package [5]. This approach accurately accounts for infinite, long-range electrostatic interactions by splitting the calculation into a short-range real space and a long-range reciprocal space, making it the preferred model for periodic systems. To use this model, set **LongRange** = [2, []] (no parameters) in the settings file.

Note: Generating the precomputed energy components is typically faster with the truncated Coulomb method than with Ewald summation, a difference that becomes more pronounced as the system size increases.

Short-range interaction (optional):

- No interaction: To turn off the short-range interaction, set **ShortRange** = [0, []] (no parameters) in the settings file.

- **Overlap:** A strong positive repulsive energy is applied to interchangeable ions positioned within a specified distance, `r_overlap` (in Å), of another ion⁸. This energetic penalty is used to exclude overlapping configurations during the sampling process. To use this model, set `ShortRange = [1, [r_overlap]]` in the settings file.

3.4.1 Energy Functions

- `get_neighbors()`: For each site, identifies neighbors and distances from them. It is called by `LR-TruncatedCoulomb()`.
- `SR_Overlap()`: Computes the short-range overlap interaction energy between two ions with an overlap radius `r_overlap`. It is optionally called by `LR-TruncatedCoulomb()` and `LR-EwaldSum()`.
- `LR-TruncatedCoulomb()`: Computes the components of the interaction energy using the truncated Coulomb model with a cutoff radius `r_cutoff` for long-range interactions. Optionally, adds a short-range interaction in addition to the long-range one. It is called by `get_PrecomputedEnergy()`.
- `LR-EwaldSum()`: Computes the components of the interaction energy using the Ewald summation model with automatically determined parameters for long-range interactions. Optionally, adds a short-range interaction in addition to the long-range one. It is called by `get_PrecomputedEnergy()`.
- `get_PrecomputedEnergy()`: Selects the long-range and the optional short-range energy models defined in *settings.jl* and generates the precomputed components of the interaction energy by calling the functions above. It is called by the function `get_data()` of the `ImportData` module.

3.5 SelectSolver

The `SelectSolver` module executes the solver defined in the settings file *settings.jl* and returns the solution tuple `Solution` generated by the solver, which stores best solution(s) found and their energies. A solution is defined as a distribution of interchangeable ions among the optimization sites. The inputs of this module are the settings tuple `Settings` (see section 3.1) and the data tuple `Data` (see section 3.2).

Note: In this version of SOSS, only one optimization solver is provided. However, the package is designed to accommodate multiple solvers, which is the purpose of this module.

⁸In other words: interchangeable-anchored and interchangeable-interchangeable pairs. This repulsive term does not apply to anchored-anchored pairs, since their relative positions do not change during optimization and, therefore, such configurations (in the event of overlap between anchored ions) could not be excluded.

3.5.1 SelectSolver Functions

- `get_params()`: Creates a parameters tuple **Params**, which stores the parameters defined in the parameters file *params.jl*. This tuple is required by the optimization solver, which is called from `solve()`.
- `solve()`: Reads **Data** and **Settings**, gets **Params**, calls the optimization solver defined in *settings.jl* and returns **Solution**.

3.6 SANSS

The **SANSS** module implements the Simulated Annealing Structure Search optimization solver, which is based on the component-based formulation of the Simulated Annealing algorithm [1]. Enriched with **SOSS**-specific components, this customized solver takes as input the data tuple **Data** (section 3.2) and the parameters tuple **Params** (section 3.5), and returns the solution tuple **Solution** (section 3.5).

In accordance with the terminology introduced in [1], a series of in-house modifications can be categorized into the following algorithmic components: **Initial Temperature**, **Temperature Length**, **Cooling Schemes**, **Neighbourhood**, **Exploration Criterion**, **Acceptance Criterion**, and **Stopping Criterion**. Additional components, not included in [1], are **Recurrence**, which allows for multiple optimizations, **Display Style**, which controls the amount of information displayed, and **Plots**, which generate optimization evolution plots. Several elements specific to the structural optimization problems addressed by **SOSS**, such as the generation of an initial solution and the definition of the **Neighbourhood**, have been implemented. The initial solution is generated by randomly distributing the interchangeable ions among the optimization sites. When the number of interchangeable ions is less than the number of optimization sites, some sites remain empty and serve as vacancies. This distribution of ions can be either fully random or restricted according to user-defined constraints specified in the settings file. Such constraints divide the optimization sites into subsets and allow each ion type to occupy only the subset assigned to it. The **Neighbourhood** of a solution is defined as the set of all solutions reachable through a single move. A move consists of either swapping the positions of an interchangeable ion and a vacancy, or swapping the positions of two interchangeable ions. When restrictions are applied, the possible moves for each ion are limited to the sites within its designated subset. In addition, a double **Cooling Scheme** has been implemented to allow reheating during the optimization. This mechanism helps the optimization escape local minima, where the search may become trapped when using a basic cooling scheme.

Table 2 summarizes the main components of the **SANSS** algorithm, while Table 3 lists and describes all **SANSS**-related parameters. Recommended parameter values, derived from extensive testing, are also provided. Users are encouraged to reuse the example parameter files supplied in the test cases as templates and modify them as needed, using Table 3 as a guide.

The implementation of **SANSS** is organized into two main functions, **SANSS_set()** and **SANSS_opt()**, from which the remaining functions, responsible for specific tasks, are called. The **SANSS_set()** function sets up the complete **SANSS** algorithm by initializing the solver, executing the optimization cycles, and collecting the results. At each optimization cycle, an initial solution is generated and the **SANSS_opt()** function, which performs the energy minimization and returns the best solution found, is called. Thus, **SANSS_set()** controls the overall optimization workflow, while **SANSS_opt()** performs the optimization of an individual initial solution. The pseudocode of the **SANSS_set()** and **SANSS_opt()** functions is provided in Algorithms 1 and 2, respectively. In addition, a description of all functions in the module is provided in section 3.6.1.

For a more conceptual description of the algorithm and its theoretical background, as well as a discussion of the recommended parameter values listed in Table 3, readers are referred to the accompanying manuscript [2].

Table 2: **SANSS** algorithmic components.

Component	Description	Options	Parameters
Initial Temperature	Sets the initial (max) and final (min) values for the reference temperature	• OIT1: The max and min values are freely chosen • OIT2: The max and min values are proportional to the max and min energy difference between two consecutive steps of a random walk (RW) performed within SANSS • OIT3: The max and min values are selected to match some max and min probability of acceptance, defined as $\text{prob.} = \exp(-\Delta E / T)$, where ΔE is the average energy difference between two consecutive steps of a RW performed within SANSS	✓ IT_scheme ✓ IT_params
Temperature Length	Controls the number of candidate moves at each reference temperature value	N/A	✓ tempLength

Continued on next page

Component	Description	Options	Parameters
Cooling Scheme	Governs the temperature updates	Main Cooling Scheme	N/A
	The proposed cooling methodology introduces (1) a Main Cooling Scheme that controls a reference temperature according to a geometric cooling scheme, and (2) a Secondary Cooling Scheme that relies on the reference temperature and the selected cooling scheme (arithmetic or constant) to compute the current temperature	Secondary Cooling Scheme • OMCS1: The reference temperature is updated according to a geometric cooling and its initial and final values, provided by the Initial Temperature component • OSCS1: The current temperature does not change; i.e., it is constant and equal to the reference temperature • OSCS2: The current temperature is updated according to an arithmetic cooling from the reference temperature	✓ scheme ✓ beta
Exploration Criterion	Chooses one candidate solution from the Neighbourhood of the current solution	• OEC1: The candidate solution is the best solution among a given number of randomly generated solutions in the Neighbourhood of the current solution • OEC2: The candidate solution is the first solution improving the current solution before the given number of solutions used in OEC1 is reached (if none improves the current solution, the best one among them is selected)	✓ gamma ✓ ScanningStyle ✓ FirstToImprove

Continued on next page

Component	Description	Options	Parameters
Acceptance Criterion	Determines whether the candidate solution generated by the Exploration Criterion is accepted. The Metropolis criterion is used (accepts a candidate solution with the probability: $\min\{1, \exp(-\Delta E/T)\}$, where ΔE is the difference between energies at the proposed and incumbent solutions)	N/A	N/A
Stopping Criterion	Controls the termination of the Simulated Annealing algorithm	• OSC1: The optimization stops after steps steps in the Main Cooling Scheme • OSC2: The optimization stops after MaxStagnation steps in the Main Cooling Scheme without improving the best solution (this option is combined with OSC1) • OSC3: The optimization stops if the temperature reaches a value lower than 1 (this stopping condition is mandatory)	✓ steps ✓ MaxStagnation
Recurrence	Controls the number of optimization cycles to be performed; i.e., the independent runs (optimizations) starting from different initial solutions (ion distributions over the optimization sites)	N/A	✓ nRepeats

Continued on next page

Component	Description	Options	Parameters
Display	Controls the amount of information to be displayed on the screen	• ODS1: Only the best solution is shown • ODS2: The evolution of best solutions found during optimization is displayed • ODS3: The summary of each step within the Main Cooling Scheme is shown	✓ DisplayStyle
Plots	Controls the generation of optimization evolution plots	N/A	✓ plots

Table 3: SANSS parameters (see Table 2 for the description of options).

Parameter Name and Data Type	Description	Recommended Values
IT_scheme ::Int64	Initial Temperature schemes: • OIT1: IT_scheme = 1 • OIT2: IT_scheme = 2 • OIT3: IT_scheme = 3	1
IT_params ::Vector{Float64}	Parameters for the Initial Temperature schemes: • OIT1: IT_params = [initial T, final T] • OIT2: IT_params = [RW steps, scale constant] • OIT3: IT_params = [RW steps, max prob., min prob.]	OIT1: [1000, 10]
tempLength ::Int64	Number of steps in the Secondary Cooling Scheme	1000 - 10000
scheme ::String	Secondary Cooling Schemes: • OSCS1: scheme = "constant" • OSCS2: scheme = "arithmetic"	"arithmetic"

Continued on next page

Parameter Name and Data Type	Description	Recommended Values
beta ::Float64	Intensity of the Secondary Cooling Scheme ($T[\text{tempLength}] = \text{beta} \times T[1]$) Note: this parameter is ignored when using OSCS1	0.1
gamma ::Int64	Max number of explored solutions before selecting the candidate solution (OEC1)	100 - 1000
ScanningStyle ::Int64	Update of the max number of explored solutions (OEC1) during the optimization: • No update: ScanningStyle = 1	1
FirstToImprove ::Bool	Control variable to activate / deactivate OEC2: • Activate: FirstToImprove = true • Deactivate: FirstToImprove = false	true
steps ::Int64	Number of steps in the Main Cooling Scheme (OSC1)	10 - 1000
MaxStagnation ::Int64	Control variable to activate / deactivate the limit of stagnation steps (OSC2): • Activate: MaxStagnation = User's Choice • Deactivate: MaxStagnation = 0	0
nRepeats ::Int64	Number of optimization cycles	1 (testing stage) ≥ 8 (production runs)
DisplayStyle ::Int64	Display Styles: • ODS1: DisplayStyle = 0 • ODS2: DisplayStyle = 1 • ODS3: DisplayStyle = 2	2 (serial runs) 0 (parallel runs)
plots ::Bool	Control variable to activate / deactivate the evolution plots: • Activate: plots = true • Deactivate: plots = false	true (serial runs) false (parallel runs)

Algorithm 1: `SANSS.set()`^a

Input: data tuple `Data`; SANSS parameters tuple `Params`

Output: solution tuple `Solution`

Initialization:

1 call `InitialTemperature()` to initialize temperatures $T_{\max}$ and $T_{\min}$ according to **Initial Temperature**^b scheme and `Data`;

Optimization:

2 for $run = 1$ to `nRepeats` do // **Recurrence**

3 | call `get_init_sol_constrained()` / `get_init_sol()` to propose an initial solution S_0 according to `Data`;

4 | call `get_nbrs_constrained()` / `get_nbrs()` to get **Neighbourhood** N_0 of solution S_0 according to `Data`;

5 | call `SANSS.opt()` to find the best solution S_{best} and its energy E_{best} ;

6 | append S_{best} , E_{best} , and run to arrays SS_{best} , EE_{best} , $runs$;

7 end

Output Management:

8 if **Plots** then

9 | call `plot_data()` to generate figures *acceptance-evol.pdf*^c, *energy-evol.pdf*,
| *best-solution.pdf*;

10 end

11 call `write_summary()` to generate file *summary.dat*;

12 save SS_{best} , EE_{best} , $runs$ into `Solution`;

13 return `Solution`

^aThe functions highlighted in magenta in Algorithms 1 and 2 are discussed in section 3.6.1.

^bThe algorithmic components shown in bold in Algorithms 1 and 2 are explained in Table 2.

^cThe files/figures presented in violet in Algorithms 1 and 2 are described in section 3.6.2.

Algorithm 2: SANSS_opt()^a

Input: data tuple *Data*; parameters tuple *Params*; initial solution S_0 and its Neighbourhood N_0 ; max temperature $T_{\max}$; min temperature $T_{\min}$
Output: best solution S_{best} and its energy E_{best}
Solution and Neighbourhood Initialization:
1 set $S_{\text{best}} = S_0$; $N_{\text{best}} = N_0$;
Solution Energies Initialization:
2 call **get_TotalEnergy()** to compute energy E_0 of S_0 ;
3 set $E_{\text{best}} = E_0$; $\Delta E_{\text{best}} = \Delta E_0 = 0$;
Temperature Initialization:
4 set temperature range of **Main Cooling Scheme** to $[T_{\min}, T_{\max}]$;
5 set $T_0 = T_{\max}$;
Cooling Schemes:
6 for $i = 1$ to steps do // **Main Cooling Scheme**
7 | call **getListT!()** to generate a list of current temperatures T_{list} according to T_0 ,
 Secondary Cooling Scheme, and Temperature Length;
8 | for $T \in T_{\text{list}}$ do // **Secondary Cooling Scheme**
9 | | call **move_simulation!()**;
10 | | call **getBestFromScanNeighSimulation()** to choose a new solution $S_1 \in N_0$ in
 accordance with **Exploration Criterion**, get its **Neighbourhood** N_1 , and
 compute the change in energy ΔE_1 from S_0 to S_1 ;
11 | | call **metropolis()** to accept/reject S_1 in accordance with
 Acceptance Criterion;
12 | | if S_1 is accepted then // **update configuration**
13 | | | set $S_0, N_0, \Delta E_0$ to $S_1, N_1, \Delta E_0 + \Delta E_1$;
14 | | end
15 | | end
16 | | if $\Delta E_0 < \Delta E_{\text{best}}$ then // **update best configuration**
17 | | | set $S_{\text{best}}, E_{\text{best}}, N_{\text{best}}, \Delta E_{\text{best}}$ to $S_0, E_0 + \Delta E_0, N_0, \Delta E_0$;
18 | | end
19 | | if **Stopping Criterion** [OSC3]^b is met then
20 | | | exit **Secondary and Main Cooling Schemes**;
21 | | end
22 | end
23 | call **returnToPreviousOpt!()** to restore S_0, N_0 , and ΔE_0 to their best values $S_{\text{best}},$
 N_{best} , and ΔE_{best} ;
24 | update T_0 in accordance with **Main Cooling Scheme**;
25 | if **Stopping Criterion** [OSC2] is met then
26 | | exit **Main Cooling Schemes**;
27 | end
28 end
Output Management:
29 call **write_data()** to generate files *acceptance_evol.dat*, *energy_evol.dat*, *best_solution.dat*;
30 return $S_{\text{best}}, E_{\text{best}}$

^aSee footnotes a, b and c in Algorithm 1 for statements in magenta, violet and bold fonts.

^bThe options [OSCx] ($x = 2, 3$) in Algorithm 2 are introduced in Table 2.

3.6.1 SANSS Functions

- `get_init_sol_constrained()`: Generates an initial random solution, i.e., a distribution of interchangeable ions over the optimization sites, while respecting the ion-site constraint (each ion type may occupy only a specific subset of optimization sites). The function returns a 2D array in which each column (second index) corresponds to a single interchangeable ion. The first index (with four possible values) specifies the stored property: (1) ion ID, (2) position in the optimization-site numbering, (3) position in the total-site numbering, and (4) ion type. It is called by `SANSS_set()`.
- `get_init_sol()`: Same as `get_init_sol_constrained()`, but without enforcing the ion-site constraint (any interchangeable ion type may occupy any optimization site). It is called by `SANSS_set()`.
- `get_nbrs_constrained()`: Reads an initial random solution generated by `get_init_sol_constrained()` and returns its **Neighbourhood** (2D array indicating the possible moves for each interchangeable ion in the solution). Two types of moves are allowed: (1) a move of an ion to a vacancy, and (2) exchanging positions of two ions. The first move is applied to subsets of optimization sites containing vacancies (style 1), whereas for subsets without vacant sites (style 2) the second move is assigned. The first index of the array identifies the subset, while the second index enumerates the neighbors associated with that subset. For style 1 subsets, the neighbors correspond to the vacant sites within the subset; for style 2 subsets, they correspond to all ions belonging to the subset. The function also returns two vectors containing the move style (1 or 2) and the number of neighbors of each subset. It is called by `SANSS_set()`.
- `get_nbrs()`: Same as `get_nbrs_constrained()`, but without the ion-site constraint. The returned 2D array contains a single row (i.e., one subset), to which all interchangeable ions and optimization sites belong. In this case, no subdivision into multiple subsets is applied. It is called by `SANSS_set()`.
- `TempRandomWalk()`: Reads an initial solution, performs a Random Walk, and returns the average, minimum and maximum energy difference between two consecutive steps to compute **Initial Temperature** when OIT2 or OIT3 (Table 2) is selected. It is called by `InitialTemperature()`.
- `InitialTemperature()`: Computes the minimum and maximum temperatures for the **Main Cooling Scheme**. Several **Initial Temperature** schemes are available (see Table 2). It is called by `SANSS_set()`.
- `get_InterInterEnergy()`: Computes the interchangeable ion pairs interaction energy of a solution using the precomputed components of the interaction energy (interchangeable-interchangeable) stored in `Data`. It is called by `get_TotalEnergy()`.
- `get_InterAnchEnergy()`: Computes the interchangeable-anchored ion pairs interaction energy of a solution using the precomputed components of the interaction energy (interchangeable-anchored) stored in `Data`. It is called by `get_TotalEnergy()`.

- `get_TotalEnergy()`: Computes the total (interchangeable ion pairs + anchored ion pairs + interchangeable-anchored ion pairs) interaction energy of a solution. The interchangeable-interchangeable and interchangeable-anchored interaction energies are computed by calling `get_InterInterEnergy()` and `get_InterAnchEnergy()`, whereas the anchored-anchored interaction energy is a constant value stored in the data tuple `Data`. It is called by `SANSS_set()`, `SANSS_opt()`, and `TempRandomWalk()`.
- `getListT!()`: Computes the temperatures list for the **Secondary Cooling Scheme**. Several schemes are available (see Table 2). It is called by `SANSS_opt()`.
- `get_dE1()`: Generates a new solution in the **Neighbourhood** of the current solution and computes the difference of energy of the move. An interchangeable ion is selected at random, then it is moved to a vacancy (style 1) or swapped with another interchangeable ion (style 2) at random, and finally, the difference of energy between the old and new configuration is computed. The function returns the move style, the solution and **Neighbourhood** updates, and the difference of energy. It is called by `getBestFromScanNeighSimulation()`.
- `getBestFromScanNeighSimulation()`: Chooses a new solution in the **Neighbourhood** of the current solution according to the **Exploration Criterion**. A number of solutions specified in *params.jl* with the parameter `gamma` is generated using `get_dE1()`, and the best solution among them is chosen. If `FirstToImprove = true`, the first solution that improves the current solution is chosen. The function returns the move style, the solution and **Neighbourhood** updates, and the difference of energy. It is called by `move_simulation!()`.
- `metropolis()`: Accepts/rejects a new solution according to the **Acceptance Criterion**. It is called by `move_simulation!()`.
- `move_simulation!()`: Calls `getBestFromScanNeighSimulation()` to choose a new solution in the **Neighbourhood** of the current solution according to the **Exploration Criterion**, and calls `metropolis()` to accept/reject the new solution according to the **Acceptance Criterion**. If the new solution is accepted, the current solution, its **Neighbourhood**, and its energy are updated using the information provided by `getBestFromScanNeighSimulation()`. It is called by `SANSS_opt()`.
- `returnToPreviousOpt!()`: If the current solution is not the best solution, returns to the previous best solution (and its associated **Neighbourhood** and energy). It is called by `SANSS_opt()`.
- `write_data()`: Generates the *acceptance_evol.dat*, *energy_evol.dat* and *best_solution.dat* files, described in section 3.6.2. It is called by `SANSS_opt()`.
- `SANSS_opt()`: Performs optimization in the SANSS algorithm (pseudocode in Algorithm 2). The function reads an initial solution, its **Neighbourhood**, and a list of SANSS parameters, and returns the best solution found together with its energy. The algorithm proceeds as follows: (1) initializes the **Main Cooling Scheme** reference temperature;

(2) generates a list of working temperatures according to the **Secondary Cooling Scheme**; (3) for each temperature, proposes a new solution according to the **Exploration Criterion**; (4) accepts or rejects the proposed solution according to the **Acceptance Criterion**, updating the current solution if accepted; (5) updates the best solution when an improvement is found; (6) updates the reference temperature and repeats steps (2)-(5) until the **Stopping Criterion** is satisfied; and (7) returns the best solution found and its energy. During the optimization, information is displayed on the screen according to the **Display Style**, and the files *acceptance_evol.dat*, *energy_evol.dat*, and *best_solution.dat* are generated (see section 3.6.2 for more details on SANSS output files). The function is called by `SANSS_set()`.

- `plot_data()`: Generates the *acceptance_evol.pdf*, *energy_evol.pdf* and *best_solution.pdf* plots, described in section 3.6.2. It is called by `SANSS_set()`.
- `write_summary()`: Generates the *summary.dat* file, described in section 3.6.2. It is called by `SANSS_set()`.
- `SANSS_set()`: Sets up a complete SANSS algorithm (pseudocode in Algorithm 1). The function reads a data tuple `Data` and a parameters tuple `Params`, and returns a solution tuple `Solution`. The algorithm proceeds as follows: (1) initializes the temperature according to **Initial Temperature**; (2) generates an initial random solution and computes its **Neighbourhood**; (3) obtains an optimal solution by calling `SANSS_opt()`; (4) appends the found solution to a list of optimal solutions; (5) repeats steps (2)-(4) until the **Recurrence** loop terminates; (6) writes a summary file and optionally (according to **Plots**) generates plots of the data collected during the optimization (*acceptance_evol.pdf*, *energy_evol.pdf*, and *best_solution.pdf*); and (7) returns `Solution` (which includes the optimal solutions and their energies).

Note: The optimization cycles loop executed within `SANSS_set()` is designed to support parallel execution of multiple runs. To take full advantage of this functionality, `SOSS` must be launched using the same number of CPUs as the desired number of optimization cycles, `nRepeats`, to invoke `nRepeats` parallel runs, as described in section 4.5.2.

3.6.2 SANSS Output Files

The solver generates an **OPTFILES** directory for a job (see section 4.5), where it creates a separate subdirectory for each run, **RUN_X** for $X = 1, \dots, nRepeats$ (number of runs of the job), along with a *summary.dat* file.

Each **RUN_X** directory contains the following files:

- *acceptance_evol.dat*: Acceptance rate evolution along the Main Cooling Scheme.
- *energy_evol.dat*: Energy evolution along the Main Cooling Scheme.
- *best_solution.dat*: Energy evolution of best solutions sampled with `SOSS`.

- *acceptance_evol.pdf*, *energy_evol.pdf*, *best_solution.pdf*: Plots of the reported data (optional).

The *summary.dat* file includes:

- Parameters used.
- Final energy for each run.
- Statistics: best run, worst run, average energy, and optimization time.

4 Using SOSS

4.1 System Requirements

The code is compatible with all major operating systems (Windows, Linux, and macOS). However, the scripts provided for automated installation and usage are designed for Unix-like command-line interfaces. For this reason, **we strongly advise using Linux or macOS operating systems**.

For Windows users, we recommend the Windows Subsystem for Linux (WSL) [6] or Cygwin [7] as alternative environments for running the package. While it is technically possible to adapt the scripts for native Windows command-line use, this requires manual modification and is not recommended, unless the user has significant experience with Windows scripting.

To install and run SOSS, the following software must be installed on a user’s computer:

- **Python**: Installation instructions are available on the official Python website [8]. The latest available version is recommended.
- **Julia**: Installation instructions are available on the official Julia website [9]. The latest available version is recommended.

A user should make sure that Python and Julia are added to the system PATH. This is usually handled automatically when the aforementioned software packages are installed through the official channels.

4.2 Unpacking the Software

The software is distributed as a gzipped tar file containing the SOSS source code, test suites for the package, and scripts for running and testing the code. Two documents, Installation and User Guide and “Screening Energetically Stable Structures in Solid-State Ionics Applications”, A. Teruel et al. are also supplied.

To unpack the software, type the following command: `tar -xzf SOSS.tar.gz`. This will create the SOSS directory shown in Figure 1, accompanied by the two above-mentioned documents.

4.3 Installation

To install SOSS and verify the installation, navigate to the `./INSTALL` directory and first run: `julia --project=. install_soss.jl`. This will perform the following steps:

1. Create a Julia project environment in the current directory (`./INSTALL`).
2. Install Julia packages listed in *Project.toml* to the project environment.
3. Install Python packages listed in *CondaPkg.toml* to the project environment.
4. Pre-compile the installed packages.

Then, run: `julia --project=. test_installation.jl` in the same directory. This will open the Julia project environment linked to the `./INSTALL` directory and try to import the packages listed in *Project.toml* and *CondaPkg.toml*. If the installation passes the test, a success message will be displayed; otherwise, errors will be shown.

For detailed information related to the installation, refer to the files in the `./INSTALL` directory, which are thoroughly documented.

4.4 Validation

To validate the correct functionality of the code and to familiarize users with its usage, two test cases are provided (**Test 1** and **Test 2**). All required input files are included, along with a script to execute both tests automatically.

The tests generate optimized undoped (**Test 1**) and doped (**Test 2**) LLZO structures. The LLZO material is widely studied in solid-state applications due to its high ionic conductivity [10–15]. The tests **Test 1** and **Test 2** are briefly described below, whereas more details can be found in [2], which is also included in the SOSS package.

Test 1: Optimization without Dopants

In this test, the composition of the material remains unchanged. The objective is to identify the optimal atomic configuration of a specific species already present in the structure. The input structure corresponds to $\text{Li}_{960}\text{La}_{192}\text{Zr}_{128}\text{O}_{768}$ ($2 \times 2 \times 2$ supercell of LLZO with all possible Li sites occupied). The optimization aims to obtain the structure $\text{Li}_{448}\text{La}_{192}\text{Zr}_{128}\text{O}_{768}$ by selecting an energetically favorable distribution of 448 Li atoms across the 960 available Li sites. The settings tags defined in *settings.jl* for this test are shown below.

```
# Optimization Solver
solver = "SANSS"

# Input and Output Atomic Structure Files Format
IN_format = "VASP"
OUT_format = "VASP"
```

```

# Initial Structure
FU_symbols = ["Li", "La", "Zr", "O"]
FU_numbers = [7.0, 3.0, 2.0, 12.0]
FU_charges = [1.0, 3.0, 4.0, -2.0]
FU_protons = [3, 57, 40, 8]
FU_copies = 64

# Host ions
Ihost_symbols = ["Li"]
Ihost_numbers = [7.0]

# Added ions / dopants
Iadd_symbols = []
Iadd_numbers = []
Iadd_charges = []
Iadd_protons = []
Iadd_sites = []

# Ion-Site constraint
IonSiteConstraint = false

# Energy model
LongRange = [1, [7.0]]
ShortRange = [1, [1.7]]

```

Although the input structure contains 15 Li sites per formula unit, only 7 Li ions are physically present in the material. The remaining sites correspond to vacant positions included in the structure solely to define the optimization sites. Consequently, `FU_numbers` must specify 7 Li ions per formula unit rather than 15. Since the optimization is performed exclusively on the Li sublattice, Li is defined as the only host ion species through `Ihost_symbols = ["Li"]`. Consequently, all Li sites present in the input structure are treated as optimization sites. The target composition contains 7 Li ions per formula unit, and therefore `Ihost_numbers = [7.0]` specifies the number of Li ions that must remain in the optimized structure. No dopants are introduced in this test. Accordingly, the tags associated with added ions (`Iadd_symbols`, `Iadd_numbers`, `Iadd_charges`, `Iadd_protons`, and `Iadd_sites`) are left empty. Finally, `IonSiteConstraint = false` because all optimization sites belong to the same chemical species and no restrictions on site occupancy are required. Finally, the energy model is defined through the `LongRange` and `ShortRange` tags. In this test, a truncated Coulomb potential with a cutoff radius of 7.0 Å is used to describe long-range electrostatic interactions. In addition, a short-range overlap term with an overlap radius of 1.7 Å is included. This term strongly penalizes configurations in which ions are separated by distances smaller than the overlap radius, thereby preventing unphysical arrangements.

Test 2: Optimization with Aliovalent Doping

This test involves a modification of the composition by introducing dopants. Starting from the same initial structure as in **Test 1**, a subset of the original atoms is removed, and new

dopant species are added. The goal is to obtain the doped structure $\text{Li}_{432}\text{La}_{192}\text{Zr}_{112}\text{Ta}_{16}\text{O}_{768}$. During optimization, 432 Li atoms are distributed among the 960 initial Li sites, while 112 Zr and 16 Ta atoms are distributed among the 128 initial Zr sites. The settings tags defined in *settings.jl* for this test are shown below.

```
# Optimization Solver
solver = "SANSS"

# Input and Output Atomic Structure Files Format
IN_format = "VASP"
OUT_format = "VASP"

# Initial Structure
FU_symbols = ["Li", "La", "Zr", "O"]
FU_numbers = [7, 3, 2, 12]
FU_charges = [1.0, 3.0, 4.0, -2.0]
FU_protons = [3, 57, 40, 8]
FU_copies = 64

# Host ions
Ihost_symbols = ["Li", "Zr"]
Ihost_numbers = [6.75, 1.75]

# Added ions / dopants
Iadd_symbols = ["Ta"]
Iadd_numbers = [0.25]
Iadd_charges = [5.0]
Iadd_protons = [73]
Iadd_sites = [2]

# Ion-Site constraint
IonSiteConstraint = true

# Energy model
LongRange = [1, [7.0]]
ShortRange = [1, [1.7]]
```

As in **Test 1**, the input structure contains 15 Li sites per formula unit, although only 7 Li ions are physically present in the material. Consequently, `FU_numbers` must specify 7 Li ions per formula unit rather than 15. In this case, the optimization is performed simultaneously on the Li and Zr sublattices. Therefore, `Ihost_symbols = ["Li", "Zr"]` defines Li and Zr as the host-ion species, and all Li and Zr sites present in the input structure are treated as optimization sites. The target composition contains 6.75 Li ions and 1.75 Zr ions per formula unit, which is specified through `Ihost_numbers = [6.75, 1.75]`. Ta is introduced as a dopant species through `Iadd_symbols = ["Ta"]`. The target composition contains 0.25 Ta ions per formula unit, and therefore `Iadd_numbers = [0.25]`. The corresponding charge and atomic number are specified through `Iadd_charges = [5.0]` and

`ladd_protons = [73]`. Unlike **Test 1**, prior knowledge about the system is incorporated through the ion-site constraint. Since Ta ions are known to substitute for Zr ions in LLZO, `IonSiteConstraint = true` is used to activate this functionality. The optimization sites are then divided into two subsets: subset 1 contains the Li sites, while subset 2 contains the Zr sites. The tag `ladd_sites = [2]` specifies that Ta ions may only occupy sites belonging to subset 2, thereby restricting Ta substitutions to the Zr sublattice. As a result, the 0.25 Ta ions and 1.75 Zr ions per formula unit are distributed among the 2.0 Zr sites available per formula unit, meaning that no vacancies are created within this subset. In contrast, the 6.75 Li ions are distributed among the 15 available Li sites per formula unit, leaving a fraction of the Li sites vacant. The same energy model as in **Test 1** is used.

To begin testing, navigate to the `./WORK` directory, which contains the following sub-directories:

- **INPUTS**
- **OUTPUTS**
- **RUN**

The content of each subdirectory is explained in section 2.3. The input files for **Test 1** and **Test 2** are located in the directories `./INPUTS/TEST_1` and `./INPUTS/TEST_2`, respectively. For both tests the same structure file is used since they share the same initial structure. Default values are used for all parameters, except for: `steps = 10` (reduced from 100 to accelerate testing), `nRepeats = 1` (instead of `nRepeats ≥ 8`, for simplicity).

To run **Test 1** navigate to the `./WORK/RUN/TEST_1` directory and run the following command: `source ../SCRIPTS/run_job.sh TEST_1 1`. During execution, the settings used, data import progress, optimization progress, and results export progress are displayed on the screen. The message `TEST_1 was executed successfully` on the screen, followed by the results location and the execution time, confirms that SOSS successfully passed **Test 1**.

To run **Test 2** navigate to the `./WORK/RUN/TEST_2` directory and run the following command: `source ../SCRIPTS/run_job.sh TEST_2 1`. Similar information as in **Test 1** is displayed on the screen. In this case, the message `TEST_2 was executed successfully` confirms that the code successfully passed **Test 2**.

On an Apple M1 Pro chip, the entire process takes approximately 1-2 minutes. The **Test 1** and **Test 2** results can be found in the directories `./WORK/OUTPUTS/TEST_1` and `./WORK/OUTPUTS/TEST_2`. Each of these directories stores the files described in section 2.3.2. Below we discuss the results the user should find in the aforementioned directories. Note that Figure 2 and Figure 3 were generated using VESTA [16], but any tool compatible with the VASP format can be used.

Figure 2 and Figure 3 present the initial and final (optimized) structures for **Test 1** and **Test 2**, respectively. In the initial structure of **Test 1** (Figure 2 (left)), all 960 Li sites are

fully occupied by Li (green), whereas in the final configuration (2 (right)) only 448 of these sites remain occupied by Li atoms, with the rest left vacant. While the initial structure of **Test 2** (Figure 3 (left)) is the same as in **Test 1**, in its final structure (Figure 3 (right)), in addition to the presence of vacancies among the Li sites (432 Li atoms remain in the final structure), the original 128 Zr sites have been redistributed between 112 Zr atoms (blue) and 16 Ta dopant atoms (yellow).

Figure 4 displays the energy evolution during the **SANSS** optimization. As expected, both tests exhibit a decreasing energy trend as the optimization progresses. This behavior would be even more pronounced with a higher number of optimization steps.

Figure 5 shows the energy evolution of best solutions sampled with **SOSS**. In **Test 1** (left), the total energy decreases by 18.25 eV/atom from the initial random configuration (initial solution) to the final optimized structure (best solution), while in **Test 2** (right) the decrease amounts to 16.50 eV/atom.

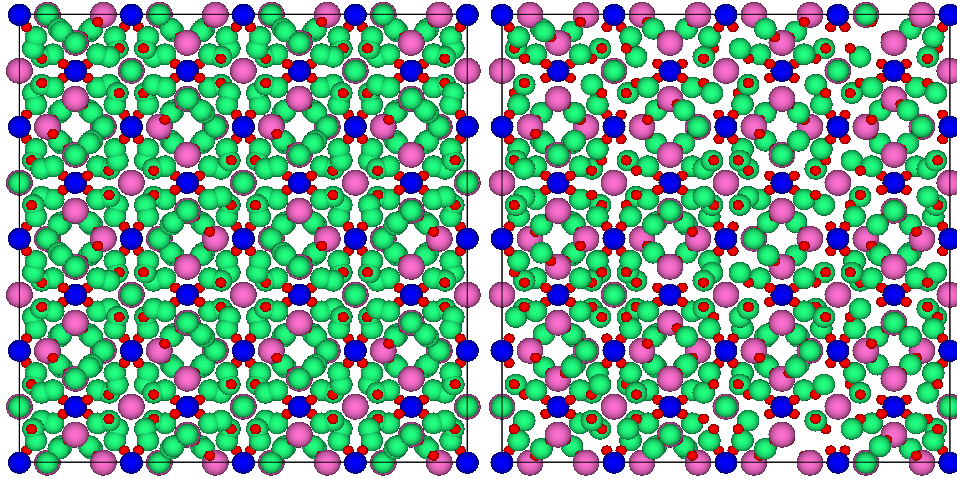


Figure 2: **Test 1** initial (left) and final (right) structures. Species: Li (green), La (pink), Zr (blue), O (red).

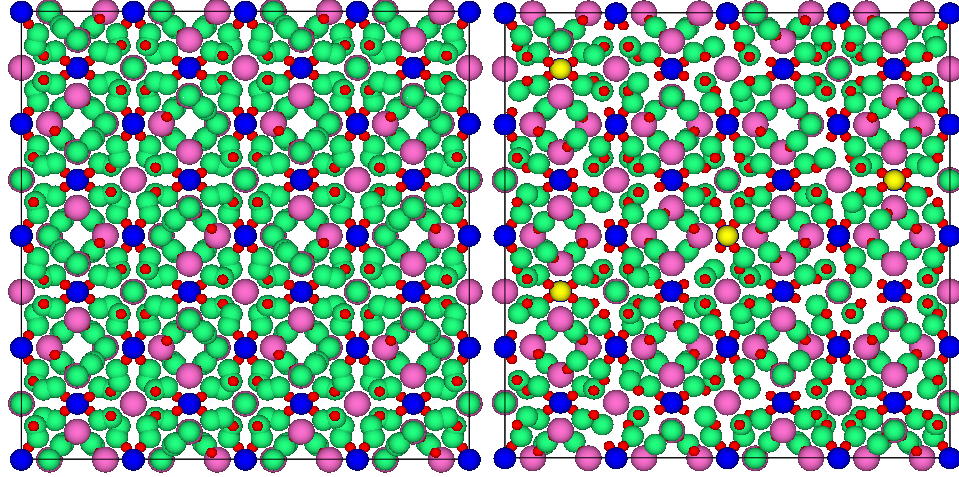


Figure 3: **Test 2** initial (left) and final (right) structures. Species: Li (green), La (pink), Zr (blue), Ta (yellow), O (red).

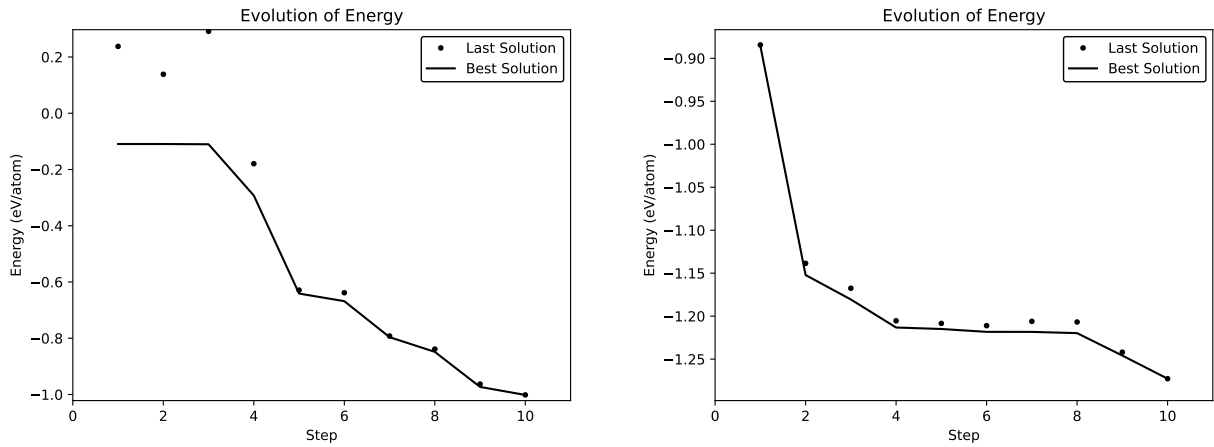


Figure 4: Evolution of energy during the **SANSS** optimization observed in **Test 1** (left) and **Test 2** (right). Plots automatically generated by **SANSS** when the parameter `plots` is set to true.

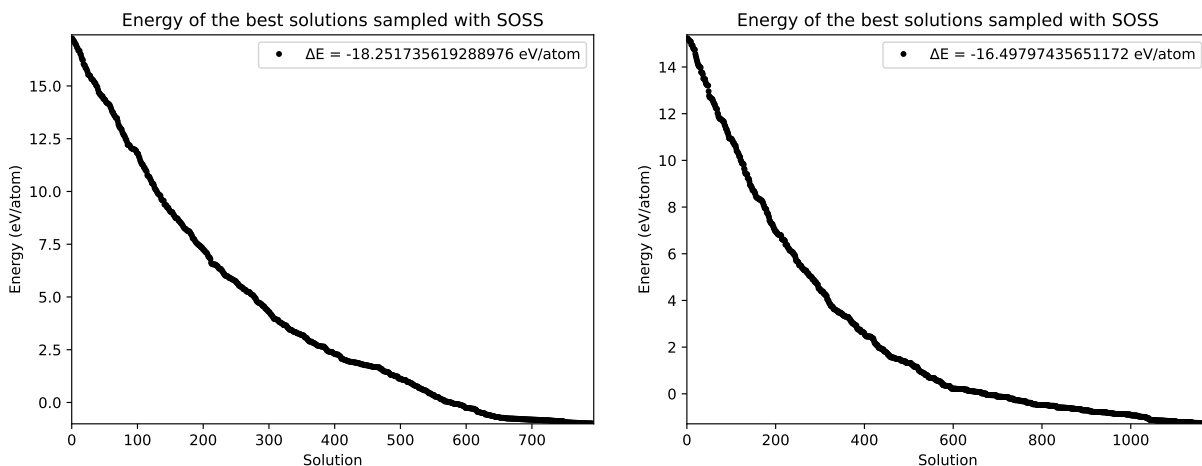


Figure 5: Energy evolution of best solutions sampled with SOSS for **Test 1** (left) and **Test 2** (right). Plots automatically generated by SANSS when the parameter `plots` is set to true.

4.5 Running SOSS

In this section we explain how to prepare and execute a job, as well as to analyze its results. The entire process is performed within the `./WORK` directory.

4.5.1 Preparing Input Files

To prepare the input files, go to the `./WORK/INPUTS` directory and follow the steps below:

1. Create a subdirectory with the name of a job (e.g., `MY_JOB`).
2. Add the structure file (`instruct.vasp`) to this new directory.
3. Copy the input files `settings.jl` and `params.jl` from any of the test directories into the new directory.
4. Optionally, add a `seeds.dat` file. One could copy this data, e.g. from any of the test directories.
5. Modify the input files (`settings.jl` and `params.jl`) as needed.
 - The `settings.jl` file is strictly dependent on the studied system, so it must be adapted to a system at hand. See Table 1 for information on settings tags.
 - The `params.jl` file is less dependent on the studied system, but since parameter values affect both optimization time and the quality of the results, they should be carefully reviewed. We recommend to refer to Table 3 for guidelines.

4.5.2 Executing Scripts

To run the job, navigate to the `./WORK/RUN` directory and create the subdirectory `MY_JOB`. Then, enter that directory and execute the script `run_job.sh` using the following command: `source ../SCRIPTS/run_job.sh MY_JOB ncpus`. Where:

- **MY_JOB** is the name of the subdirectory in **./WORK/INPUTS**, created by the user.
- **ncpus** is the number of CPUs to use. It is recommended to use the same number of CPUs as the number of optimization cycles **nRepeats** specified in the parameter file (*params.jl*).

Note: All CPUs must be within the same node, as currently, the package is not intended for use in a distributed computing environment.

The script will:

1. Copy the *run_soss.jl* script from **./WORK/RUN/SCRIPTS** to **./WORK/RUN/MY_JOB**.
2. Copy the input files from **./WORK/INPUTS/MY_JOB** to **./WORK/RUN/MY_JOB**.
3. Perform the optimization for those inputs (using the *run_soss.jl* script with **ncpus** CPUs).
4. Create the **./WORK/OUTPUTS/MY_JOB** directory and save the results there.
5. Clean the **./WORK/RUN/MY_JOB** directory.

The message **MY_JOB was executed successfully** on the screen followed by the execution time and the results directory indicates that the optimization finished successfully.

4.5.3 Examining Output Files

To examine the results, go to the **./WORK/OUTPUTS/MY_JOB** directory and follow these steps:

1. For analysis of the optimization process, open the **OPTFILES** directory.
 - It generally contains files that monitor the evolution of some properties (e.g., energy and acceptance) during the optimization process. See section 3.6.2 for more details.
2. For analysis of the final (optimal) structures, open the **STRUCTURES** directory.
 - For each optimization cycle/run, one will find a file named *outstruct.x.vasp* containing the final structure in VASP format (POSCAR).
 - These files can be visualized or analyzed using any tool that supports the VASP format.

References

- [1] A. Franzin, T. Stützle, Revisiting simulated annealing: A component-based analysis, *Comput. Oper. Res.* 104 (2019) 191–206. doi:10.1016/j.cor.2018.12.015.
- [2] A. Teruel, M. R. Bonilla, H. A. Cortés, C. León, J. Carrasco, R. Melnik, E. Akhmatkaya, Screening energetically stable structures in solid-state ionics applications, *Comput. Phys. Commun.*
- [3] Viena Ab initio Simulation Package (VASP) Wiki, POSCAR, <https://vasp.at/wiki/index.php/POSCAR>. Accessed: 22/09/2025.
- [4] P. P. Ewald, Die Berechnung optischer und elektrostatischer Gitterpotentiale, *Ann. Phys.* 369 (1921) 253–287. doi:10.1002/andp.19213690304.
- [5] S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, G. Ceder, Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis, *Comput. Mater. Sci.* 68 (2013) 314–319. doi:10.1016/j.commatsci.2012.10.028.
- [6] Windows Subsystem for Linux (WSL) official website, <https://learn.microsoft.com/en-us/windows/wsl/>. Accessed: 22/09/2025.
- [7] Cygwin official website, <https://www.cygwin.com/>. Accessed: 22/09/2025.
- [8] Python official website, <https://www.python.org/>. Accessed: 22/09/2025.
- [9] Julia official website, <https://julialang.org/>. Accessed: 22/09/2025.
- [10] R. Murugan, V. Thangadurai, W. Weppner, Fast Lithium Ion Conduction in Garnet-Type $\text{Li}_7\text{La}_3\text{Zr}_2\text{O}_{12}$, *Angew. Chem. Int. Ed.* 46 (2007) 7778–7781. doi:10.1002/anie.200701144.
- [11] J. Awaka, N. Kijima, H. Hayakawa, J. Akimoto, Synthesis and structure analysis of tetragonal $\text{Li}_7\text{La}_3\text{Zr}_2\text{O}_{12}$ with the garnet-related type structure, *J. Solid State Chem.* 182 (2009) 2046–2052. doi:10.1016/j.jssc.2009.05.020.
- [12] M. R. Bonilla, A. Lozano, B. Escribano, J. Carrasco, E. Akhmatkaya, Revealing the Mechanism of Sodium Diffusion in Na_xFePO_4 Using an Improved Force Field, *J. Phys. Chem. C* 122 (2018) 8065–8075. doi:10.1021/acs.jpcc.8b00230.
- [13] M. R. Bonilla, F. A. García Daza, J. Carrasco, E. Akhmatkaya, Exploring Li-ion conductivity in cubic, tetragonal and mixed-phase Al-substituted $\text{Li}_7\text{La}_3\text{Zr}_2\text{O}_{12}$ using atomistic simulations and effective medium theory, *Acta Mater.* 175 (2019) 426–435. doi:10.1016/j.actamat.2019.06.033.
- [14] S. Kumazaki, Y. Iriyama, K.-H. Kim, R. Murugan, K. Tanabe, K. Yamamoto, T. Hirayama, Z. Ogumi, High lithium ion conductive $\text{Li}_7\text{La}_3\text{Zr}_2\text{O}_{12}$ by inclusion of both al and si, *Electrochem. Commun.* 13 (2011) 509–512. doi:10.1016/j.elecom.2011.02.035.

- [15] S. Adams, R. P. Rao, Ion transport and phase transition in $\text{Li}_{7-x}\text{La}_3(\text{Zr}_{2-x}\text{M}_x)\text{O}_{12}$ ($\text{M} = \text{Ta}^{5+}, \text{Nb}^{5+}$, $x = 0, 0.25$), J. Mater. Chem. 22 (2012) 1426–1434. doi:10.1039/c1jm14588f.
- [16] K. Momma, F. Izumi, VESTA 3 for three-dimensional visualization of crystal, volumetric and morphology data, Applied Crystallography 44 (2011) 1272–1276. doi:10.1107/S0021889811038970.